

Projet 2 : Génération de maillage pour le calcul scientifique

Antoine Levrat

04/2026

L'objectif de ce travail est d'implémenter un algorithme de Delaunay pour une triangulation 2D et l'appliquer à un cas de compression d'image. Certains résultats et codes se basent sur le Projet 1 effectué précédemment.

Nous allons d'abord nous placer dans le carré test $[0, 1]^2$. L'objectif dans un premier temps est de placer des points aléatoirement dans ce carré et les rajouter à la triangulation.

1 Noyau simplifié

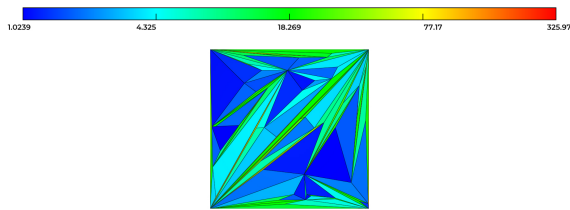
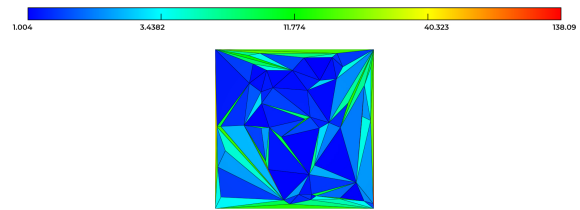
Ce premier algorithme se base sur le "edge flip". Il consiste à rajouter un point P , trouver à quel triangle il appartient, créer 3 triangles soit 3 nouvelles arêtes ayant pour sommet le nouveau point P . Puis pour chaque côté de chaque triangle créé, tester la qualité des triangles avec la fonction qualité Q1 ou Q2 du premier projet. On effectue ensuite un edge flip. On recalcule la qualité, en comparant avec les valeurs précédentes on garde la configuration qui avait la meilleure qualité.

On notera qu'il est important de faire attention à bien réactualiser les voisins, sur ces étapes. De plus nous avons choisi, comme dans le sujet srand(7) pour avoir une génération de point reproductible.

Note d'implémentation : La fonction edge flip ne marche que quand les 2 triangles forment un convexe. Pour vérifier ceci, dans un quadrilatère (A, B, C, D) , pour une arête (B, C) on vérifie que le point D est à gauche de (A, B) et le point C à gauche de (A, D) *i.e*) les aires signées de (A, B, D) et (A, D, C) sont positives. (Un dessin permet de s'en assurer)

Par un raisonnement similaire, pour trouver le triangle dans lequel est P on cherche le seul triangle du maillage où l'aire signée est positive pour tous les côtés. On implémente d'abord un algorithme simple qui teste tous les triangles, puis par la suite on a implémenté un algorithme qui suit les directions négatives.

On peut donc visualiser les résultats obtenus : Sur un carré $[0, 1]^2$ avec 50 points insérés aléatoirement via srand(7)

**Figure 1** – Sans Edge Flip**Figure 2** – Avec Edge Flip

la solution montre la qualité Q1 pour chaque triangle. Les images sont visualisées avec VIZIR4, en appliquant une échelle log sur la palette.

On observe que le flip permet d'améliorer grandement la qualité des triangles.

2 Noyau de Delaunay

On essaie maintenant de mettre en place un noyau de Delaunay. L'algorithme se met en place via plusieurs étapes :

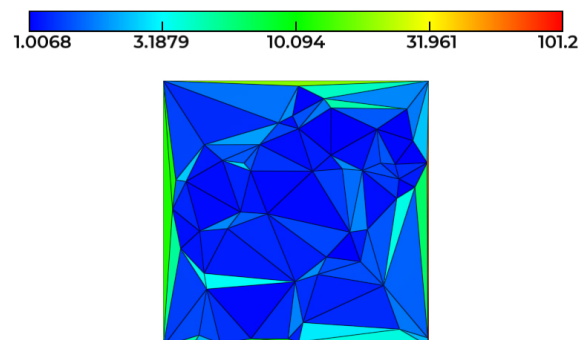
1. Localiser le triangle dans lequel est situé le nouveau point P .
2. Calculer la cavité associée à P et en retirer les triangles intérieurs.
3. Créer les triangles en étoilant autour de P .

Pour savoir quels triangles il faut retirer, on effectue le critère de la sphère : *on test : le point P appartient-il au cercle inscrit du triangle testé ?*

Nous avons dans un premier temps testé ce critère sur tout les triangles du maillages pour simplifier l'implémentation. Par la suite, on s'est réservé de la fonction qui localise P , on récupère ensuite les voisins du triangle trouvé, on effectue le test de la sphère sur chacun, et on réitère sur leurs voisins jusqu'à ce que tous les voisins respectent le critère.

Vis à vis de la suppression des triangles on a choisi de récupérer les arêtes frontières de la cavité, puis d'étoiler P . On réutilise les anciens emplacements des anciens triangles et on en recrée si besoin.

On donne ici les résultats dans la même configuration que dans les Figures précédentes. *i.e)* Sur un carré $[0, 1]^2$ avec 50 points insérés aléatoirement via `rand(7)`

**Figure 3** – Noyau de Delaunay

On visualise ici aussi le log de la qualité Q1 de chaque triangle. On voit que la qualité des triangles créés est bien meilleure restant inférieurs à 3 pour la grande majorité des triangles créés.

Par ailleurs on décide de comparer le temps CPU entre les 2 noyau :

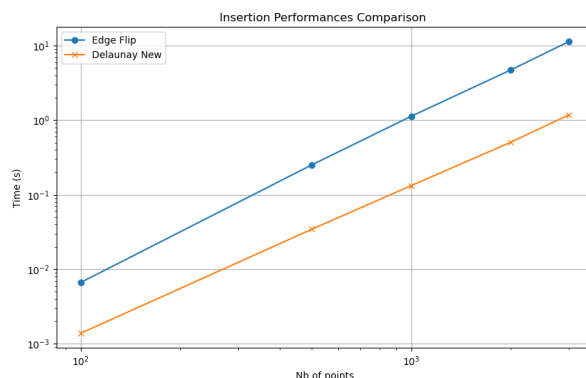


Figure 4 – Comparaison du temps par rapport au nombre de points insérés pour noyau Edge Flip et Delaunay

Le graphique compare en échelle logarithmique le temps passé pour un nombre de points entre 100 et 3000. On observe que pour 3000 le Edge Flip passe 11 secondes, tandis que Delaunay passe un peu plus de 2 secondes.

L'objectif de ce travail est d'implémenter un algorithme de Delaunay pour une triangulation 2D et l'appliquer à un cas de compression d'image. Certains résultats et codes se basent sur le Projet 1 effectué précédemment.

Nous allons d'abord nous placer dans le carré test $[0, 1]^2$. L'objectif dans un premier temps est de placer des points aléatoirement dans ce carré et les rajouter à la triangulation.

3 Noyau simplifié

Ce premier algorithme se base sur le "edge flip". Il consiste à rajouter un point P , trouver à quel triangle il appartient, créer 3 triangles soit 3 nouvelles arêtes ayant pour sommet le nouveau point P . Puis pour chaque côté de chaque triangle créé, tester la qualité des triangles avec la fonction qualité Q1 ou Q2 du premier projet. On effectue ensuite un edge flip. On recalcule la qualité, en comparant avec les valeurs précédentes on garde la configuration qui avait la meilleure qualité.

On notera qu'il est important de faire attention à bien réactualiser les voisins, sur ces étapes. De plus nous avons choisi, comme dans le sujet srand(7) pour avoir une génération de point reproductible.

Note d'implémentation : La fonction edge flip ne marche que quand les 2 triangles forment un convexe. Pour vérifier ceci, dans un quadrilatère (A, B, C, D) , pour une arête (B, C) on vérifie que le point D est à gauche de (A, B) et le point C à gauche de (A, D) *i.e*) les aires signées de (A, B, D) et (A, D, C) sont positives. (Un dessin permet de s'en assurer)

Par un raisonnement similaire, pour trouver le triangle dans lequel est P on cherche le seul triangle du maillage où l'aire signée est positive pour tous les côtés. On implémente

dabord un algorithme simple qui teste tout les triangles, puis par la suite on à implémenté un algorithme qui suis les directions negatives.

On peut donc visualiser les resultats obtenus : Sur un carré $[0, 1]^2$ avec 50 points insérés aléatoirement via `srand(7)`

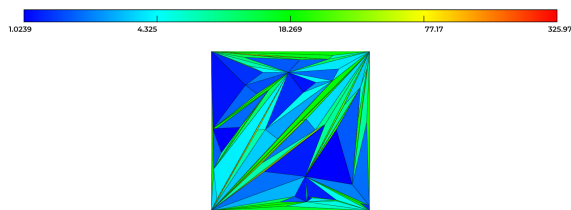


Figure 5 – Sans Edge Flip

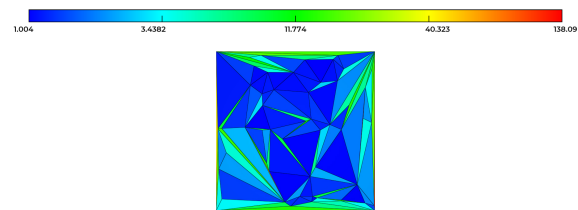


Figure 6 – Avec Edge Flip

la solution montre la qualité Q1 pour chaque triangle. Les images sont visualisées avec VIZIR4, en appliquant une echelle log sur la palette.

On observe que le flip permet d'améliorer grandement la qualité des triangles.

4 Noyau de Delaunay

On essaie maintenant de mettre en place un noyau de Delaunay. L'algorithme ce met en place via plusieurs étapes :

1. Localiser le triangle dans lequel est situé le nouveau point P .
2. Calculer la cavité associée a P et en retirer les triangles interieurs.
3. Créer les triangles en étoilant autour de P .

Pour savoir quels triangles il faut retirer, on effectue le critère de la sphère : *on test : le point P appartient-il au cercle inscrit du triangle testé ?*

Nous avons dans un premier temps testé ce critère sur tout les triangles du maillages pour simplifier l'implémentation. Par la suite, on s'est reservi de la fonction qui localise P , on récupère ensuite les voisins du triangle trouvé, on effectue le test de la sphère sur chaqu'un, et on réitère sur leurs voisins jusqu'a ce que tout les voisins respectent le critère.

Vis à vis de la suppression des triangles on a choisi de récupérer les arrêtes frontières de la cavité, puis d'étoiler P . On réutilise les anciens emplacements des anciens triangles et on en recrée si besoin.

On donne ici les resultats dans la même configuration que dans les Figures précédentes. *i.e)* Sur un carré $[0, 1]^2$ avec 50 points insérés aléatoirement via `srand(7)`

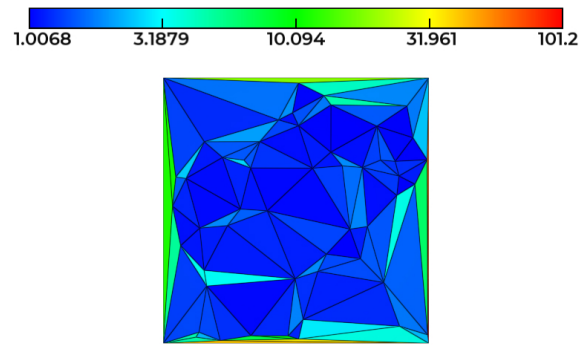


Figure 7 – Noyau de Delaunay

On visualise ici aussi le log de la qualité Q1 de chaque triangle. On voit que la qualité des triangles créés est bien meilleure restant inférieurs à 3 pour la grande majorité des triangles créés.

Par ailleurs on décide de comparer le temps CPU entre les 2 noyau :

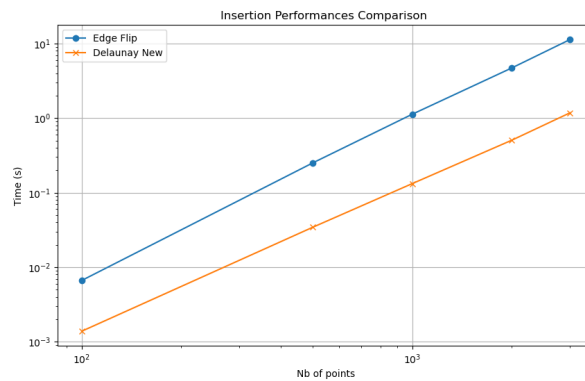


Figure 8 – Comparaison du temps par rapport au nombre de points insérés pour noyau Edge Flip et Delaunay

Le graphique compare en échelle logarithmique le temps passé pour un nombre de points entre 100 et 3000. On observe que pour 3000 le Edge Flip passe 11 secondes, tandis que Delaunay passe un peu plus de 2 secondes.